

DSA

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

- ★ 1. Introduction to Data Structures & Algorithms
- ★ 2. Time and Space Complexity (Big-O Notation)
- ★ 3. Arrays
- ★ 4. Linked Lists
- ★ 5. Stacks
- ★ 6. Queues
- ★ 7. Recursion
- ★ 8. Searching - Linear Search and Binary Search
- 9. Sorting - Bubble, Selection, and Insertion Sort
- ★ 10. Sorting - Merge Sort and Quick Sort
- ★ 11. Trees - Binary Trees
- ★ 12. Binary Search Trees (BST)
- ★ 13. Graphs
- ★ 14. Graph Traversal - BFS and DFS
- ★ 15. Hashing / Hash Tables
- 16. Heaps and Priority Queues
- ★ 17. Dynamic Programming
- 18. Greedy Algorithms

★ 1. Introduction to Data Structures & Algorithms [IMPORTANT TOPIC]

Explanation:

A Data Structure is a way of organizing and storing data so it can be used efficiently — like an array, a list, or a tree. An Algorithm is a step-by-step set of instructions to solve a problem, like sorting a list or searching for an item.

DSA is one of the most important subjects in computer science — it directly affects how fast and efficient a program is, and it is a core topic in almost every coding interview.

Key Points:

- Good DSA knowledge helps write faster, more efficient programs.
- Choosing the RIGHT data structure for a problem can make code thousands of times faster.
- Heavily tested in technical/coding interviews at most companies.

Example Code:

```
# A simple example: finding the largest number in a list
numbers = [4, 9, 2, 7, 1]
largest = max(numbers)
print(largest)
```

Output:

9

★ 2. Time and Space Complexity (Big-O Notation) [IMPORTANT TOPIC]

Explanation:

Time Complexity measures how the RUNNING TIME of an algorithm grows as the input size grows. Space Complexity measures how much extra MEMORY it needs. Big-O notation is the standard way to describe this growth, like $O(n)$, $O(\log n)$, or $O(n^2)$.

Key Points:

- $O(1)$ -> Constant time — same speed no matter the input size (example: accessing an array by index).
- $O(n)$ -> Linear time — grows directly with input size (example: looping through a list once).
- $O(\log n)$ -> Logarithmic time — very efficient, grows slowly (example: binary search).
- $O(n^2)$ -> Quadratic time — grows quickly, often from nested loops (example: bubble sort).

Example Code:

```
# O(n) - runs once for each item in the list
for num in numbers:
    print(num)
```

Output:

```
4
9
2
7
1
```

 *Note: When comparing two algorithms, lower Big-O growth is almost always better for large inputs, even if the code looks more complex.*

★ 3. Arrays [IMPORTANT TOPIC]

Explanation:

An array stores multiple values of the same type together, in a fixed, ordered sequence. It is the most basic data structure, and every value can be accessed instantly using its index.

Example Code:

```
numbers = [10, 20, 30, 40]
print(numbers[0])      # instant access, O(1)
numbers.append(50)      # add to the end
print(numbers)
```

Output:

```
10
[10, 20, 30, 40, 50]
```

 *Note: Reading an array by index is very fast ($O(1)$), but inserting or deleting in the MIDDLE is slow ($O(n)$) because other items must shift.*

★ 4. Linked Lists [IMPORTANT TOPIC]

Explanation:

A Linked List stores data as a chain of 'nodes', where each node holds a value AND a pointer/reference to the NEXT node. Unlike arrays, linked lists don't need continuous memory, and inserting/deleting is often faster.

Example Code:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

# Creating a simple linked list: 10 -> 20 -> 30
first = Node(10)
first.next = Node(20)
first.next.next = Node(30)

print(first.value, first.next.value, first.next.next.value)
```

Output:

10 20 30

 *Note: Unlike an array, a linked list has NO index — to reach the 5th item, you must walk through all the nodes before it.*

★ 5. Stacks [IMPORTANT TOPIC]

Explanation:

A Stack follows LIFO order — 'Last In, First Out'. The last item added is always the FIRST one removed, just like a stack of plates — you always take the top plate first.

Key Points:

- push() -> add an item to the top
- pop() -> remove the item from the top
- peek() -> look at the top item without removing it

Example Code:

```
stack = []
stack.append(10)    # push
stack.append(20)
stack.append(30)

print(stack.pop())  # removes and returns 30 (last in)
print(stack)
```

Output:

```
30
[10, 20]
```

 *Note: Stacks are used for tasks like the 'undo' feature in apps, and for checking balanced brackets in code.*

★ 6. Queues [IMPORTANT TOPIC]

Explanation:

A Queue follows FIFO order — 'First In, First Out'. The FIRST item added is the first one removed, just like a real queue (line) of people waiting.

Key Points:

- enqueue() -> add an item to the back
- dequeue() -> remove the item from the front

Example Code:

```
from collections import deque

queue = deque()
queue.append(10)    # enqueue
queue.append(20)
queue.append(30)

print(queue.popleft())    # removes and returns 10 (first in)
```

Output:

10

 *Note: Queues are used for tasks like handling print jobs in order, or managing requests on a server.*

★ 7. Recursion [IMPORTANT TOPIC]

Explanation:

Recursion is when a function calls ITSELF to solve a smaller version of the same problem. Every recursive function needs a 'base case' — a stopping condition — or it will run forever.

Example Code:

```
def factorial(n):  
    if n == 0:        # base case  
        return 1  
    return n * factorial(n - 1)  
  
print(factorial(4))
```

Output:
24

★ 8. Searching - Linear Search and Binary Search *[IMPORTANT TOPIC]*

Explanation:

Linear Search checks EVERY item one by one until it finds a match — simple, but slow for large lists ($O(n)$). Binary Search is much faster ($O(\log n)$), but only works on a SORTED list — it repeatedly cuts the search area in half.

Example Code:

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([10, 20, 30, 40, 50], 40))
```

Output:

3

9. Sorting - Bubble, Selection, and Insertion Sort

Explanation:

These are simple sorting algorithms, good for learning the basics of sorting, though slower ($O(n^2)$) for large lists. Bubble Sort repeatedly swaps adjacent items if they're in the wrong order. Selection Sort repeatedly finds the smallest remaining item and places it correctly.

Example Code:

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([5, 2, 9, 1, 5]))
```

Output:

```
[1, 2, 5, 5, 9]
```

★ 10. Sorting - Merge Sort and Quick Sort *[IMPORTANT TOPIC]*

Explanation:

Merge Sort and Quick Sort are much faster, more efficient sorting algorithms ($O(n \log n)$), widely used in real programming languages' built-in sort functions. Merge Sort splits the list in half, sorts each half, then merges them back together. Quick Sort picks a 'pivot' value and rearranges the list around it.

Example Code:

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return sorted(left + right)    # simplified merge step  
  
print(merge_sort([5, 2, 9, 1, 5]))
```

Output:

```
[1, 2, 5, 5, 9]
```

 *Note: Most programming languages' built-in sort() functions actually use an optimized version of Merge Sort or Quick Sort internally.*

★ 11. Trees - Binary Trees [IMPORTANT TOPIC]

Explanation:

A Tree is a data structure made of connected 'nodes', starting from one 'root' node, branching out into 'children'. A Binary Tree is a tree where each node has AT MOST two children — usually called the 'left' and 'right' child.

Example Code:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

root = Node(10)
root.left = Node(5)
root.right = Node(15)

print(root.value, root.left.value, root.right.value)
```

Output:

10 5 15

★ 12. Binary Search Trees (BST) [IMPORTANT TOPIC]

Explanation:

A Binary Search Tree is a special Binary Tree where, for every node: all values in the LEFT subtree are SMALLER, and all values in the RIGHT subtree are LARGER. This special ordering makes searching very fast ($O(\log n)$ on average).

Example Code:

```
# BST property example:
#       10
#      /  \
#     5    15
#
# Searching for 15: since 15 > 10, we only check the RIGHT side
# – we never need to look at the left side at all.
```

Output:

(This structure lets searching skip large parts of the tree, making it fast.)

★ 13. Graphs [IMPORTANT TOPIC]

Explanation:

A Graph is a set of 'nodes' (also called vertices) connected by 'edges'. Unlike trees, graphs can have cycles and multiple connections between nodes, making them perfect for modeling things like maps, social networks, and websites.

Example Code:

```
# A simple graph represented using a dictionary
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A'],
    'D': ['B']
}

print(graph['A'])    # A is connected to B and C
```

Output:

```
['B', 'C']
```

 *Note: A social network (like 'friends' connections) and Google Maps (roads between locations) are both real-world examples of graphs.*

★ 14. Graph Traversal - BFS and DFS [IMPORTANT TOPIC]

Explanation:

Traversal means visiting every node in a graph or tree in some order. BFS (Breadth-First Search) explores level by level, visiting all neighbors first. DFS (Depth-First Search) goes as DEEP as possible down one path before backtracking.

Example Code:

```
def bfs(graph, start):  
    visited = [start]  
    queue = [start]  
    while queue:  
        node = queue.pop(0)  
        print(node)  
        for neighbor in graph[node]:  
            if neighbor not in visited:  
                visited.append(neighbor)  
                queue.append(neighbor)  
  
bfs(graph, 'A')
```

Output:

```
A  
B  
C  
D
```

★ 15. Hashing / Hash Tables *[IMPORTANT TOPIC]*

Explanation:

A Hash Table stores data as key-value pairs, and uses a special 'hash function' to instantly calculate WHERE to store or find each value. This makes lookups extremely fast, usually $O(1)$ on average — Python's dictionary is built using this exact idea.

Example Code:

```
student_marks = {"Pravendra": 90, "Aman": 85}

print(student_marks["Pravendra"])    # instant lookup,  $O(1)$  on average
```

Output:

90

16. Heaps and Priority Queues

Explanation:

A Heap is a special tree-based structure where the smallest (or largest) value is ALWAYS at the top, making it very fast to repeatedly find the minimum or maximum. A Priority Queue is built using a heap, so items are processed by their PRIORITY, not just their arrival order.

Example Code:

```
import heapq

numbers = [5, 1, 8, 2]
heapq.heapify(numbers)    # turns the list into a min-heap

print(heapq.heappop(numbers))    # always removes the SMALLEST value
```

Output:

```
1
```

★ 17. Dynamic Programming [IMPORTANT TOPIC]

Explanation:

Dynamic Programming (DP) is a technique for solving complex problems by breaking them into smaller overlapping subproblems, and **STORING** their results so they are never calculated twice. This can turn an extremely slow solution into a very fast one.

Example Code:

```
# Fibonacci using Dynamic Programming
def fibonacci(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci(n - 1, memo) + fibonacci(n - 2, memo)
    return memo[n]

print(fibonacci(10))
```

Output:

55

 *Note: Without storing results (memo), calculating Fibonacci recursively becomes extremely slow for larger numbers, due to repeated work.*

18. Greedy Algorithms

Explanation:

A Greedy Algorithm makes the BEST possible choice at each step, WITHOUT worrying about the overall final result — hoping that these small good choices lead to a good overall solution. It's fast, but doesn't always give the perfect answer for every problem.

Example Code:

```
# Example: giving minimum coins for a given amount (Greedy approach)
def min_coins(amount, coins=[1, 5, 10, 25]):
    coins.sort(reverse=True)
    count = 0
    for coin in coins:
        count += amount // coin
        amount %= coin
    return count

print(min_coins(41))
```

Output:

4